

LogicRAG: A Logic-Driven Framework for Document Generation with Retrieval-Augmented Generation

Anonymous ACL submission

Abstract

Retrieval-Augmented Generation (RAG) has been widely applied to knowledge-intensive tasks, where external knowledge is retrieved to support the generation of factually accurate documents. However, existing RAG methods mainly rely on vector similarity of content for retrieval. They lack the ability to explicitly model the overall writing logic of a document, making it difficult to induce domain-specific writing logic from historical documents and provide explicit logic guidance for LLM generation. To address these limitations, we formulate the Logic-Driven Document Generation Problem (LDP), which unifies the modeling of domain writing logic and document generation within a single RAG framework. For this problem, we propose LogicRAG¹, which models the process of RAG as a logic-matching problem in a DFA state space and enables explicit retrieval and execution of writing logic. This shifts RAG’s paradigm from similarity-based content retrieval to logic-matching generation. To evaluate LogicRAG, we construct and release FinLDP-Bench², a benchmark dataset covering financial research reports with stable domain writing logic, achieving relative gains from 9.1% to 142.1% across a wide range of evaluation metrics, such as ROUGE-L and ACC.

1 Introduction

Large Language Models (LLMs) have demonstrated strong capabilities in natural language understanding and generation, especially in long-text generation and complex reasoning (Brown et al., 2020; Wei et al., 2022), but still suffer from hallucination in knowledge-intensive scenarios (Huang et al., 2025; Maynez et al., 2020). Retrieval-Augmented Generation (RAG) (Lewis

et al., 2020) mitigates this problem by incorporating external knowledge during inference and has been widely applied to open-domain question answering (Kwiatkowski et al., 2019), knowledge-intensive reasoning (Petroni et al., 2021), and multi-hop fact verification (Yang et al., 2018). Researchers have introduced it into finance, advancing applications such as quantitative investment strategy report generation (Chen et al., 2021; Kou et al., 2025), risk management and compliance report writing (Kim et al., 2023; Wu et al., 2025), corporate financial analysis report generation (Kim et al., 2024; Fatouros et al., 2024), and macroeconomic assessment (Wu et al., 2023; Lopez-Lira and Tang, 2026), showing substantial practical value.

However, domain-specific documents generation requires not only retrieving relevant facts but also strictly following the writing logic of a specific good cases. Financial research reports typically follow a highly domain-specific logical structure. For example, when the user query is “write a precious metals research report for March 2, 2025” with good cases of well history reports, the system should generate a report conforming to the target broker’s professional writing logic and patterns, usually covering sections such as *Investment Highlight*, *Price Information*, and *Risk Warning* in order, while all prices, price changes, inventories, and macro indicators must come from the user-specified query date or the corresponding reporting period, as shown in Figure 1. Existing RAG methods mainly rely on vector similarity to retrieve related content chunks, fail to learn and follow the writing logic of a specific documents, which can cause the generated report to deviate from the target writing pattern.

To address this limitation, we propose a novel research problem called the Logic-Driven Document Generation Problem (LDP) for RAG. Given a user query, sample documents (usually the index content in RAG), and a heterogeneous data source, LDP

¹Code is publicly available at: <https://anonymous.4open.science/r/LogicRAG-D267/>

²Dataset is publicly available at: <https://anonymous.4open.science/r/FinLDP-Bench-6E26/>

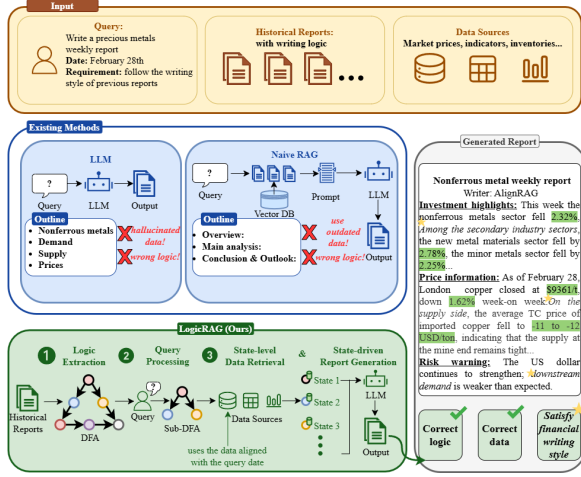


Figure 1: Workflow of Automated Generation for Precious Metals Weekly Reports

aims to generate a complete report that satisfies three requirements: (1) the generated text should follow the writing logic of historical documents; (2) the generated report should include the necessary data and information as reflected in the sample documents; (3) the data used in the generated text should be consistent with the real data.

To solve LDP, this paper proposes **LogicRAG**, which models writing logic of historical documents as a deterministic finite automaton (DFA): each state corresponds to a discourse segment with a stable semantic function, and state transitions describe the organizational order among discourse segments. During generation, LogicRAG matches and extracts a task-related sub-automaton from the global DFA according to the user query, and then retrieves the required data and generates text state by state along the transition order. In this way, LogicRAG transforms similarity-based content retrieval and generation paradigm of RAG into a writing-logic-based process of retrieval and generation, as shown in Figure 1.

To systematically evaluate LDP, we construct and release FinLDP-Bench, a benchmark dataset derived from five financial domains. We build 10,000 samples for each domain, 50,000 samples in total. Based on this dataset, we systematically compare LogicRAG with multiple strong baselines. Experimental results show that LogicRAG achieves a 142.1% improvement over the second-best method in accuracy (ACC). In addition, experiments on RotoWire (Wiseman et al., 2017) further show that LogicRAG’s state-transition generation mechanism is also applicable to structured data-to-text genera-

tion scenarios beyond financial reports.

The main contributions of this paper are as follows:

- It is the first time to propose Logic-Driven Document Generation Problem (LDP) for RAG, which unifies automatic induction of domain writing logic and document generation under logic within single framework.
- For LDP, we introduce LogicRAG, a novel approach that captures domain-specific writing logic from historical documents by formalizing it as a DFA. During generation, the model retrieves relevant sub-automaton fragments and produces text by adhering to the logical structure encoded in these automaton.
- To systematically evaluate the proposed LDP problem, we construct FinLDP-Bench, a benchmark dataset covering five financial domains with 50,000 samples in total.
- Extensive experiments on FinLDP-Bench demonstrate that LogicRAG outperforms all baselines on every evaluation metric, achieving empirical gains between 9.1% (ROUGE-L) and 142.1% (ACC).

2 Related Work

Query Optimization for Improved Retrieval Quality Methods such as HyDE (Gao et al., 2023) and Query2Doc (Wang et al., 2023) expand the original query by generating hypothetical answers or pseudo-documents; RAG-Fusion (Rackauckas, 2024) and RQ-RAG (Chan et al., 2024) improve retrieval robustness for complex questions through multi-query rewriting, decomposition, and disambiguation; MemoRAG (Qian et al., 2025), FLARE (Jiang et al., 2023), and ITER-RETGEN (Shao et al., 2023) iterative retrieval-generation into the generation process.

Retrieval-Augmented Methods with Explicit Knowledge Organization Methods such as GraphRAG (Edge et al., 2024), LightRAG (Guo et al., 2024), and GRAG (Hu et al., 2025) introduce knowledge graphs or graph structures into retrieval; LinearRAG (Zhuang et al., 2026), TG-RAG (Li et al., 2025), HopRAG (Liu et al., 2025), and HippoRAG (Jimenez Gutierrez et al., 2024) further optimize hierarchical graph representation, temporal evolution modeling, multi-hop subgraph retrieval,

and graph reasoning mechanisms; KGP (Wang et al., 2024) and SGRAG (Xie et al., 2026) enhance knowledge retrieval under complex conditional constraints.

Verification and Correction for Generation Reliability HiRAG (Huang et al., 2025) and HeteRAG (Yang et al., 2025) model hierarchical document indexes and heterogeneous knowledge fragments, respectively; Self-RAG (Asai et al., 2024) introduces a self-reflection mechanism; FActScore (Min et al., 2023) decomposes long-form factuality evaluation into atomic proposition verification.

Benchmarks Related to LDP Current datasets for RAG datasets (Kwiatkowski et al., 2019; Yang et al., 2018) mainly evaluate factual retrieval and answering, while structured data-to-text datasets such as RotoWire (Wiseman et al., 2017), ToTTo (Parikh et al., 2020), and LogicNLG (Chen et al., 2020) focus on generating text from structured records or tables.

Existing RAG methods retrieve content chunks based on semantic similarity or factual relevance, without explicitly modeling the writing logic that structures documents. Meanwhile, existing benchmarks also lack historical full-document texts with extractable writing logic, making them insufficient for evaluating LDP. Therefore, we propose LogicRAG and construct FinLDP-Bench.

3 Preliminaries and Problem Definition

This section introduces the core notation and formal definitions used in this paper. All symbols are summarized in Appendix A.

Definition: Deterministic Finite Automaton (DFA) A deterministic finite automaton L is a five-tuple:

$$L = (S, \Sigma, \delta, s_0, F) \quad (1)$$

where S is the finite set of states and Σ is the set of state-transition conditions; $\delta : S \times \Sigma \rightarrow S$ is the state-transition function; $s_0 \in S$ is the initial state, and $F \subseteq S$ is the set of final states. In this paper, the DFA is used as the formal carrier of domain writing logic, and its concrete construction from historical documents is introduced in Section 4.2.

Problem Definition: Logic-Driven Document Generation Problem (LDP) Let q be a user

query, $\mathcal{H} = \{h_i\}_{i=1}^{N_h}$ be a sample document collection containing historical reports, and $\mathcal{D} = \{d_i\}_{i=1}^{N_d}$ be a heterogeneous data source collection. LDP aims to generate a complete report $\hat{Y}$ conditioned on $(q, \mathcal{H}, \mathcal{D})$, where $\hat{Y}$ should satisfy three requirements: **logic consistency**, meaning that $\hat{Y}$ should follow the writing logic reflected in $\mathcal{H}$; **data sufficiency**, meaning that $\hat{Y}$ should cover key data and facts required by the task; and **data accuracy**, meaning that the numerical values and temporal information in $\hat{Y}$ should accurately reflect the real data in $\mathcal{D}$ and correctly correspond to the query q .

4 Method

4.1 System Overview

To address the LDP problem, inspired by the unique property of DFA that guarantees deterministic state transitions unattainable by other approaches, we propose LogicRAG, which models the writing logic of historical documents as a DFA. This ensures a unique execution path while enabling efficient traversal of discourse segments. It further frames document generation as a logic-matching and execution problem. Given a user query q , a sample document collection $\mathcal{H}$, a heterogeneous data source collection $\mathcal{D}$, a frequency threshold θ , and a similarity threshold τ , LogicRAG outputs a complete report $\hat{Y}$ that satisfies logic consistency, data sufficiency, and data accuracy. The overall architecture of LogicRAG is shown in Figure 2, and its execution process can be expressed as:

$$\begin{aligned} \hat{Y} &= \text{LogicRAG}(q; \mathcal{H}, \mathcal{D}, \theta, \tau) \\ &= \Phi(q, \mathcal{D}, \mu(q, \eta(\mathcal{H}, \theta), \tau)) \end{aligned} \quad (2)$$

LogicRAG operates in three stages, as shown in Algorithm 1. The first stage is **Document Logic Extraction** (Lines 1–4): corresponding to module η , the system extracts discourse segments with stable semantic functions from the sample document collection $\mathcal{H}$, models them as states in the deterministic finite automaton L , identifies transition relations among states, and filters low-frequency states and low-confidence transitions according to the frequency threshold θ , thereby constructing the global automaton L and the state index $\mathcal{I}$. The second stage is **Query Processing** (Lines 5–8): corresponding to module μ , the system parses the user query q , matches task-relevant states from the global automaton L , and constructs the query-specific sub-automaton L_q according to the similar-

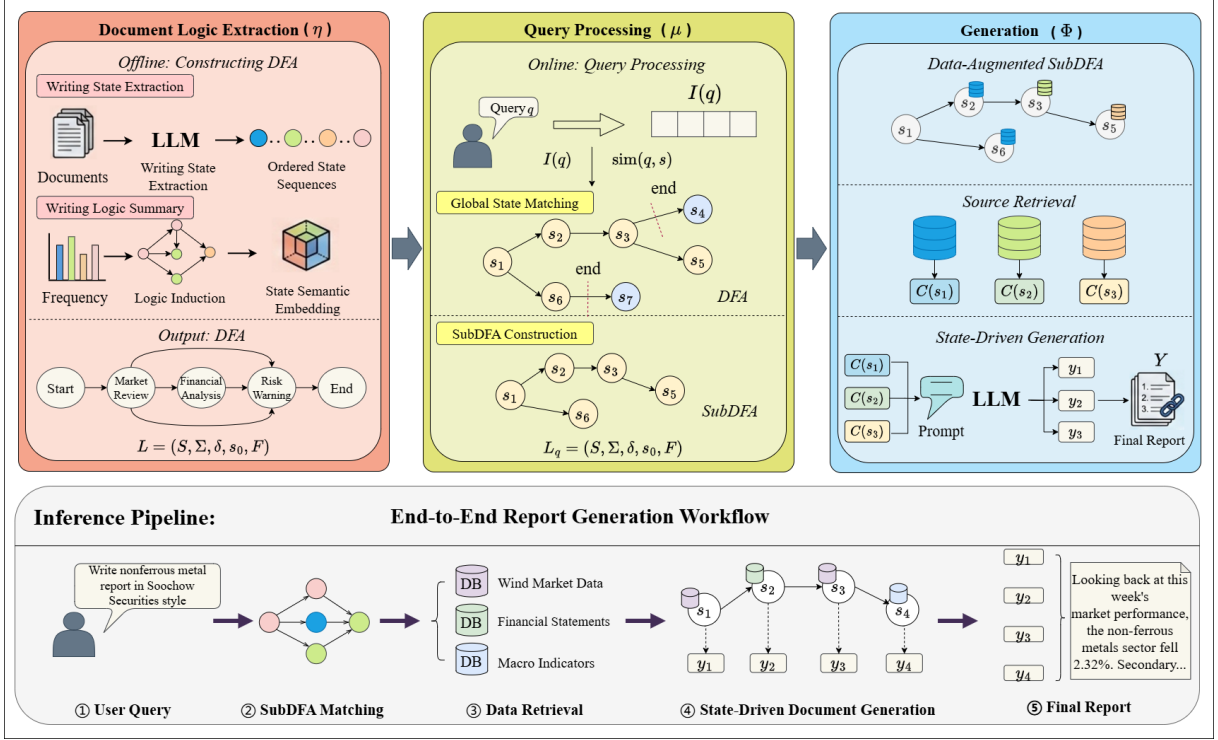


Figure 2: Overview of the LogicRAG framework, which reframes financial document generation as logic-guided generation rather than content-centric retrieval.

Algorithm 1 LogicRAG: High-Level Execution Overview

Input: Sample document collection $\mathcal{H}$, user query q , data source collection $\mathcal{D}$, frequency threshold θ , similarity threshold τ

Output: Generated document $\hat{Y}$

- 1: $L \leftarrow \text{CONSTRUCTDFA}(\mathcal{H}, \theta)$
- 2: // η : extract states and transition relations from $\mathcal{H}$
- 3: // filter low-frequency states and transitions under threshold θ and build DFA L
- 4: // compute state embeddings and build index structure $\mathcal{I}$
- 5: $L_q \leftarrow \text{BUILDSUBDFA}(q, L, \mathcal{I}, \tau)$
- 6: // μ : encode q and identify the root state
- 7: // retain connected states and prune low-similarity branches under threshold τ
- 8: // output the task-specific sub-automaton L_q
- 9: $\hat{Y} \leftarrow \text{GENERATEBYTRANSITION}(L_q, q, \mathcal{D})$
- 10: // Φ : query data sources $\mathcal{D}$ according to q and the states in L_q
- 11: // generate local text segments along the transitions in L_q
- 12: // concatenate all generated segments in logical order
- 13: **return** $\hat{Y}$

ity threshold τ and the connectivity among states. The third stage is **Generation** (Lines 9–12): corresponding to module Φ , the system executes L_q by traversing its states and transition function. At each visited state, LogicRAG retrieves the required evidence from the heterogeneous data source collection $\mathcal{D}$ according to q and the data requirements of the current state, generates the corresponding local text segment, and then moves to the next state following the transitions in L_q . Finally, the state-level outputs produced along the execution path are integrated into the complete report $\hat{Y}$.

4.2 Document Logic Extraction

This section introduces the implementation of module η . LogicRAG organizes the sample document collection $\mathcal{H}$ as a logic-based database, where stable discourse segments are extracted from historical documents and modeled as states in the deterministic finite automaton. The detailed algorithm of η is given in Algorithm 3 in Appendix C.

Let n historical documents form the collection $\mathcal{H} = \{h_1, h_2, \dots, h_n\}$. For each document h_i , η divides its natural text into ordered discourse segments with stable semantic functions and models each segment as a state. Let the j -th state in document h_i be s_i^j , with $s_i^j \in S$. Its quadruple form is

defined as

$$s_i^j = (\text{id}(s_i^j), \text{label}(s_i^j), \text{desc}(s_i^j), \text{data}(s_i^j)) \quad (3)$$

where $\text{id}(s_i^j)$ denotes the unique identifier, $\text{label}(s_i^j)$ denotes the semantic label, $\text{desc}(s_i^j)$ denotes the content summary, and $\text{data}(s_i^j)$ denotes the heterogeneous data requirement of the state. When no data requirement exists, we set $\text{data}(s_i^j) = \emptyset$. The available heterogeneous data sources are organized in $\mathcal{D}$, whose detailed definition is given in Appendix B.

For document h_i , all extracted states form an ordered state sequence, denoted as $\text{Seq}(h_i) = [s_i^1, \dots, s_i^j]$. Since different reports may use different section names and local expressions, original states cannot be directly aligned across documents. Therefore, η vectorizes state labels $\text{label}(s_i^j)$ and merges semantically similar states through nearest-neighbor retrieval, obtaining the global state sequence $\text{Seq}(\mathcal{H}) = [s^1, s^2, \dots, s^j]$. Then, η induces the shared writing logic by counting the occurrence frequency of states and adjacent state pairs. Here, $\theta \in (0, 1]$ denotes the frequency filtering threshold. For a state $s \in \text{Seq}(\mathcal{H})$ and any state pair (s_a, s_b) , their occurrence frequency and transition frequency are defined as follows:

$$\text{freq}(s) = \frac{|\{h_i \in \mathcal{H} : s \in \text{Seq}(h_i)\}|}{|\mathcal{H}|} \quad (4)$$

$$\text{freq}(s_a \rightarrow s_b) = \frac{|\{h_i \in \mathcal{H} : \exists j, s_i^j = s_a \wedge s_i^{j+1} = s_b\}|}{|\mathcal{H}|} \quad (5)$$

On this basis, η filters global states and transitions using threshold θ to construct the deterministic finite automaton L . For each retained transition (s^a, s^b) , the corresponding mapping label σ_{ab} is stored in Σ . The initial state s_0 is the state in S with the highest occurrence frequency and no incoming edge, while the final-state set $F \subseteq S$ consists of states with no outgoing edge. Figure 3 illustrates the DFA construction process before and after frequency-based filtering.

To support online semantic matching, η jointly encodes the label and content summary of each retained state $s^j \in S$:

$$e(s^j) = \text{Embed}(\text{label}(s^j) \oplus \text{desc}(s^j)) \in \mathbb{R}^{\mathcal{F}} \quad (6)$$

where $\oplus$ denotes text concatenation and $\mathcal{F}$ denotes the embedding dimension. Finally, η builds a FAISS-HNSW index $\mathcal{I}$ (Malkov and Yashunin,

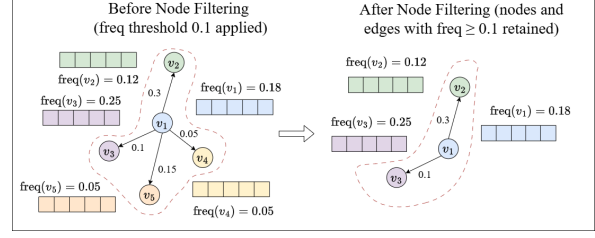


Figure 3: Illustration of the DFA construction process before and after frequency-based filtering.

2020) over all state embeddings for online query retrieval. The prompt for extracting the original state sequence is given in Appendix D.

4.3 Query Processing

This section introduces the implementation of module μ . LogicRAG models query processing as a logic-matching problem over a deterministic finite automaton. The detailed algorithm of μ is given in Algorithm 4 in Appendix C.

Given a user query q , μ first performs intent parsing and extracts a structured intent vector $I(q)$ through an encoder to represent key task information such as report type and time range. The detailed extraction method for $I(q)$ is given in Appendix D. After obtaining the intent vector $I(q)$, μ reads the embedding representation of each state from the state index $\mathcal{I}$ and computes the similarity between each state $s^j \in S$ and the query intent. The system first performs global state filtering according to threshold τ , retaining only states satisfying $\text{sim}(I(q), e(s^j)) > \tau$. This condition selects logic states that are relevant to the current query and filters out task-irrelevant states before sub-automaton construction.

After obtaining the query-relevant states, μ aims to construct a connected and executable sub-automaton L_q from the global automaton L for the current task. Specifically, μ first identifies a suitable root state s_{0q} based on the hierarchical parent-child relations in F , so that the selected root can cover the relevant logic states. If multiple candidates exist, the state with the highest similarity to the query intent is selected as the root. Starting from s_{0q} , μ then expands reachable states along the global transition function δ until the retained relevant states are covered, thereby obtaining the state set S_q . Based on S_q , μ extracts the corresponding transitions, transition conditions, and final states from the global automaton, denoted as δ_q , Σ_q , and F_q , respectively. Finally, these components are in-

tegrated to form the query-specific sub-automaton L_q . The overall construction process is shown in Figure 4.

$$L_q = (S_q, \Sigma_q, \delta_q, s_{0q}, F_q) \quad (7)$$

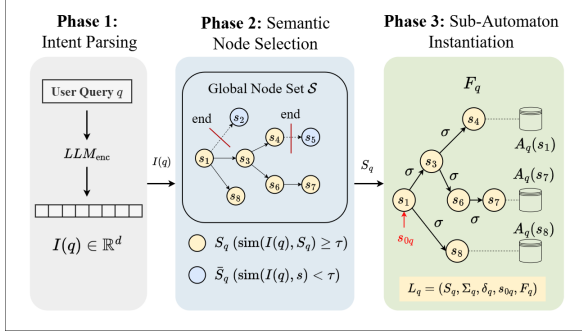


Figure 4: Overview of the sub-automaton construction process, where the user query is matched to the global logic automaton to derive a task-specific sub-automaton.

4.4 Generation

This section introduces the implementation of module Φ . Φ takes the user query q and the sub-automaton L_q output by μ as input, and finally outputs the complete financial research report $\hat{Y}$. LogicRAG uses states in the sub-automaton L_q as basic execution units and performs data retrieval and text generation state by state. The detailed algorithm of Φ is given in Algorithm 5 in Appendix C.

For any state $s^j \in S_q$, Φ constructs the retrieval conditions for the data corresponding to that state based jointly on $\text{data}(s^j)$ and the time information contained in query q (the retrieval method is described in Appendix B). On this basis, Φ searches the heterogeneous data source collection $\mathcal{D}$ for the database or API interface that satisfies the requirements of the current state, and executes the corresponding data query. The retrieval function is defined as:

$$\text{Retrieve}(\mathcal{D}, q, \text{data}(s^j)) \mapsto C(s^j) \quad (8)$$

where $C(s^j)$ denotes the set of state-data instances corresponding to state s^j . After data retrieval is completed, Φ generates document content state by state according to the transition order specified by the sub-automaton L_q . To avoid continuously accumulating the full historical output in the prompt while maintaining semantic coherence between adjacent states, the system generates a brief summary after each state text segment and uses this summary as the local contextual input for the next state's

generation. Specifically, let the initial summary be $m^0 = \emptyset$. For each state $s^j \in S_q$, the output text of the current state is defined as

$$\hat{y}^j = \text{LLM}(q, s^j, C(s^j), m^{j-1}) \quad (9)$$

where q characterizes the topic and specific requirements of the current document generation task, s^j constrains the local writing task of the current generated text, $C(s^j)$ provides factual evidence for the current generated text, and m^{j-1} denotes the summary of the previous state's generated content, which supplies compact contextual information for the current generation. After obtaining the current state's text segment $\hat{y}^j$, the system further generates the corresponding summary m^j . This summary is not included in the final document concatenation. Instead, it serves as contextual input for the next state's generation. The prompt design used for state-driven generation is given in Appendix D. After all states in the sub-automaton have completed generation, LogicRAG integrates the state texts according to the transition order and obtains the final report.

4.5 Time Complexity Analysis

This section analyzes the time complexity of LogicRAG from the offline to online stages.

Theorem 4.1 (Offline Time Complexity of LogicRAG). *Let $|\mathcal{H}|$ denote the number of historical documents, T_h denote the average cost of extracting states and constructing the local logic structure for a single article, N_s denote the total number of original states extracted from all historical documents, $\mathcal{F}$ denote the state embedding dimension, and $|S|$ denote the number of global states retained after filtering by the frequency threshold θ . The overall time complexity of the offline stage of LogicRAG is*

$$T_{\text{offline}} = O(|\mathcal{H}|T_h + N_s\mathcal{F} \log N_s + N_s + |S|\mathcal{F} \log |S|) \quad (10)$$

Theorem 4.2 (Online Time Complexity of LogicRAG). *Let $|S|$ denote the total number of states in the deterministic finite automaton, $|S_q|$ denote the number of states in the sub-automaton L_q , T_{ret} denote the average data retrieval cost for a single state, and T_{LLM} denote the average LLM generation cost for a single state. The overall time complexity of the online stage of LogicRAG is*

$$T_{\text{online}} = O(|S|(\mathcal{F} + 1) + |S_q|T_{\text{ret}} + |S_q|T_{\text{LLM}}) \quad (11)$$

In summary, the offline stage of LogicRAG mainly consists of automaton construction cost, while the online stage grows linearly with the number of global states $|S|$ and the number of sub-automaton states $|S_q|$. Proofs of Theorem 4.1 and Theorem 4.2 are given in Appendix E.

5 Experimental Study

5.1 Datasets

Since most existing RAG benchmarks target general tasks such as open-domain question answering or fact verification and rarely provide structured texts from which writing logic can be explicitly extracted, this study constructs and releases FinLDP-Bench (Financial Logic-Driven Document Generation Benchmark), a financial benchmark dataset for logic-driven document generation. The dataset covers five representative categories of financial analyst reports and is organized into structured records with predefined fields. Based on the original reports from each report category, we construct 10,000 samples per category, for a total of 50,000 samples across five categories. Detailed statistics of the FinLDP-Bench are shown in Table 1. Dataset construction process is shown in Appendix F.

Appendix G provides a single-report example to illustrate the sample characteristics of FinLDP-Bench, showing its requirement for fine-grained numerical grounding. In addition to FinLDP-Bench, Appendix H reports generalization experiments on RotoWire, where LogicRAG achieves the best overall performance and demonstrates generalization for structured data-to-text generation.

5.2 Experimental Setting

Environment Configuration. All experiments are conducted on a local workstation running Ubuntu 22.04 LTS, equipped with a 24-core, 32-thread Intel Core i9-14900HX processor and an NVIDIA GeForce RTX 4060 GPU (driver version 571.96, CUDA 12.8).

Model Configuration. This study uses DeepSeek-V3 as the base generation model for LogicRAG. By default, two historical reports from the same vertical domain are used as reference inputs for the logic induction stage η , which achieves the best overall performance in the robustness experiments (see Table 21). All vectorization tasks use the text-embedding-v3 model, with the embedding dimension set to $F = 1536$ and the HNSW index parameter set to $M = 16$.

Baselines. To comprehensively evaluate the generation performance of LogicRAG, we compare it with two categories of baselines. The first category consists of retrieval-augmented generation methods, including representative **NaiveRAG** (Lewis et al., 2020), **GraphRAG** (Edge et al., 2024), **HeterRAG** (Yang et al., 2025), **RAG-Fusion** (Rackauckas, 2024), **Self-RAG** (Asai et al., 2024), and **ITER-RETGEN** (Shao et al., 2023). The second category consists of LLM models, including DeepSeek-V3, Doubao 2.0, Qwen2.5-Max, and GPT-4o.

Evaluation Metrics. This study evaluates generated documents using both qualitative and quantitative metrics. **Fluency** measures the naturalness and stylistic quality of language expression, while **Consistency** assesses whether the generated report follows domain-specific writing logic and structural organization. **ACC** evaluates the accuracy of numerical data, **BLEU-P** measures precision-oriented n-gram matching, **ROUGE-L** captures reference-content coverage and sequential alignment, and **F1** summarizes the overall text matching quality. Details are given in Appendix I.

5.3 Experimental Results Analysis

Overall Performance Analysis. As shown in Table 2, LogicRAG achieves the best performance on all metrics. Specifically, LogicRAG reaches **54.8** in Fluency and **62.0** in Consistency, with Consistency improving by **11.1%** over the best baseline Self-RAG, mainly because the global writing-logic induction and query-specific sub-automaton provide explicit constraints on discourse organization. For quantitative metrics, LogicRAG achieves an ACC of **72.4**, improving by **142.1%** over GPT-4o, where the gain comes from state-wise evidence retrieval that grounds numerical information for each logic state. BLEU-P, ROUGE-L, and F1 reach **32.3**, **41.8**, and **36.4**, respectively, benefiting from state-level generation that improves content coverage and reference-text matching.

Ablation Analysis. As shown in Table 3, removing any core component leads to performance degradation. Without the DFA module for inducing writing logic, Fluency drops from **54.8** to **30.2**, indicating that the learned automaton structure is crucial for organizing domain-specific discourse. Removing Query & SubDFA construction mainly reduces Consistency from **62.0** to **54.6**, showing that query-specific sub-automata help select task-relevant writing states and avoid structural mismatch. Without

Subset	AvgTok	MaxTok	AvgF	MaxF	AvgP	Date Range	Institution
Agriculture	637.1	884	64.7	83	4.00	2017.04–2026.01	Pacific Securities Co., Ltd.
Cotton	291.8	424	51.7	88	4.80	2023.11–2026.01	CMB Futures Co., Ltd.
Macro	354.4	422	52.8	64	6.75	2025.07–2026.01	Ping An Securities Co., Ltd.
Precious Metals	397.0	507	36.1	55	3.53	2021.01–2025.12	Soochow Securities Co., Ltd.
ETF	204.9	287	28.3	37	5.59	2024.04–2026.01	Maigao Securities Co., Ltd.
Total	404.2	884	45.9	88	4.38	2017.04–2026.01	–

Table 1: Statistics of FinLDP-Bench, including text length, numerical-field density, paragraph count, temporal coverage, and source institution of each subset.

Table 2: Overall performance comparison on FinLDP-Bench (the best results are shown in bold, and the second-best results are underlined).

Method	Fluency	Consistency	ACC	BLEU-P	ROUGE-L	F1
NaiveRAG	42.5	42.0	15.3	17.8	26.1	21.1
Self-RAG	50.7	<u>55.8</u>	20.2	25.1	31.0	27.7
ITER-RETGEN	43.9	46.4	19.8	19.9	32.6	24.7
RAG-Fusion	50.5	48.2	17.3	23.9	<u>38.3</u>	29.4
GraphRAG	<u>54.2</u>	53.3	24.9	24.9	36.2	29.5
HeteRAG	<u>54.2</u>	53.6	20.3	<u>29.3</u>	<u>38.3</u>	<u>33.2</u>
DeepSeek-V3	48.0	47.7	19.3	8.8	15.0	11.1
Qwen2.5-Max	50.3	45.0	21.2	11.8	20.5	15.0
GPT-4o	50.7	54.5	<u>29.9</u>	12.8	23.7	16.6
Doubao 2.0	47.2	47.5	17.6	15.4	23.8	18.7
LogicRAG	54.8	62.0	72.4	32.3	41.8	36.4
Relative Gain	+1.1%	+11.1%	+142.1%	+10.2%	+9.1%	+9.6%

Data Extraction, ACC drops sharply from **72.4** to **29.4**, suggesting that data retrieval is essential for reference-text matching.

Table 3: Results of ablation experiments on the financial domain structured text generation dataset.

Methods	Fluency	Consistency	ACC	BLEU-P	ROUGE-L	F1
w/o DFA	30.2	56.2	59.4	22.9	24.4	23.6
w/o Query & SubDFA	53.5	54.6	68.9	28.7	37.2	32.4
w/o Data Extraction	30.3	48.9	29.4	19.4	32.9	22.7
LogicRAG (Ours)	54.8	62.0	72.4	32.3	41.8	36.4

Efficiency Analysis. In the offline stage, LogicRAG requires an average construction time of **66.98s**, which is lower than GraphRAG’s **77.57s**. Although this one-time cost is higher than lightweight retrieval baselines, the constructed writing-logic automaton can be reused for subsequent queries, making the trade-off acceptable for complete report generation (see Appendix J).

We also evaluate LogicRAG on RotoWire, where it achieves the best ACC of **92.30** and ROUGE-L of **27.23**, demonstrating that its state-transition generation mechanism can generalize beyond financial report generation (see Appendix H).

5.4 Scalability Analysis

Table 4 shows LogicRAG’s performance under different LLMs. LogicRAG with DeepSeek-V3 achieves the best overall text-matching perfor-

mance, while GPT-4o obtains the highest Consistency and ACC at **63.7** and **80.3**, respectively. Thus, this paper adopts DeepSeek-V3 as the default backbone. Additional results in Appendix L show that $k = 2$ achieves the best balance, with **72.4** in ACC, **32.3** in BLEU-P, and **41.8** in ROUGE-L.

Table 4: Sensitivity analysis of LogicRAG across LLM backbones on financial text generation.

Methods	Fluency	Consistency	ACC	BLEU-P	ROUGE-L	F1
Doubao 2.0	52.9	54.2	62.9	28.3	37.7	32.4
GPT-4o	51.2	63.7	80.3	24.5	35.2	29.0
Qwen2.5-Max	47.0	42.2	43.0	16.2	23.7	19.3
DeepSeek-V3	54.8	62.0	72.4	32.3	41.8	36.4

6 Conclusion

We introduce a novel research problem LDP in RAG for domain-specific document generation under writing-logic. To solve it, we propose LogicRAG, a logic-guided RAG framework that models historical writing logic as a DFA and performs retrieval and generation. We construct FinLDP-Bench for systematic LDP evaluation across five financial domains, which is the first dataset for evaluating logic-driven document generation in financial RAG. Experiments demonstrate that LogicRAG consistently outperforms strong baselines.

Limitations and Ethical Considerations

LogicRAG is mainly evaluated in the financial domain. Its performance may vary when applied to other domains with different writing logic and data requirements. Moreover, wrong or outdated external data may lead to seemingly professional but incorrect reports and mislead user. Broader cross-domain evaluation and expert verification remain necessary when practical deployment.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *Proceedings of the International Conference on Learning Representations (ICLR 2024)*.

Tom Brown, Benjamin Mann, Nick Ryder, et al. 2020. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*.

Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. 2024. RQ-RAG: Learning to Refine Queries for Retrieval Augmented Generation. In *Proceedings of the First Conference on Language Modeling (COLM 2024)*.

Wenhu Chen, Jianshu Chen, Yu Su, Zhiyu Chen, and William Yang Wang. 2020. Logical Natural Language Generation from Open-Domain Tables. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7929–7942.

Zhiyu Chen, Wenhu Chen, Charese Smiley, et al. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A GraphRAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*.

Georgios Fatouros, Kostas Metaxas, John Soldatos, and Dimosthenis Kyriazis. 2024. Transforming Sentiment Analysis in the Financial Domain with ChatGPT. *Machine Learning with Applications*, 14:100508.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 1762–1777.

Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint arXiv:2410.05779*.

Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2025. GRAG: Graph Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4145–4157.

Haoyu Huang, Yongfeng Huang, Junjie Yang, Zhenyu Pan, Yongqiang Chen, Kaili Ma, Hongzhi Chen, and James Cheng. 2025. Retrieval-Augmented Generation with Hierarchical Knowledge. *Findings of the Association for Computational Linguistics: EMNLP 2025*.

Lei Huang, Weijiang Yu, Weitao Ma, et al. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(2):1–55.

Zhengbao Jiang, Frank Xu, Luyu Gao, et al. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992.

Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *Advances in Neural Information Processing Systems*, 37:59532–59569.

Alex G. Kim, Maximilian Muhn, and Valeri V. Nikolaev. 2023. From Transcripts to Insights: Uncovering Corporate Risks Using Generative AI. *Chicago Booth Research Paper No. 23-19; Fama-Miller Working Paper; University of Chicago, Becker Friedman Institute for Economics Working Paper No. 2023-132*.

Alex G. Kim, Maximilian Muhn, and Valeri V. Nikolaev. 2024. Financial Statement Analysis with Large Language Models. *Withdrawn arXiv preprint arXiv:2407.17866*.

Zhizhuo Kou, Holam Yu, Junyu Luo, Jingshu Peng, Xujia Li, Chengzhong Liu, Juntao Dai, Lei Chen, Sirui Han, and Yike Guo. 2025. Automate Strategy Finding with LLM in Quant Investment. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 18517–18533.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.

Dong Li, Yichen Niu, Ying Ai, Xiang Zou, Biqing Qi, and Jianxing Liu. 2025. T-GRAG: A Dynamic GraphRAG Framework for Resolving Temporal Conflicts and Redundancy in Knowledge Retrieval. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 11880–11889.

692	Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. 2025. HopRAG: Multi-Hop Reasoning for Logic-Aware Retrieval-Augmented Generation. In <i>Findings of the Association for Computational Linguistics: ACL 2025</i> , pages 1897–1913.	749
693		750
694		751
695		752
696		753
697		
698	Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. In <i>Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing</i> , pages 2511–2522.	754
699		755
700		756
701		757
702		758
703		759
704	Alejandro Lopez-Lira and Yuehua Tang. 2026. Can ChatGPT Forecast Stock Price Movements? Return Predictability and Large Language Models. <i>Journal of Financial Economics</i> , forthcoming.	760
705		761
706		762
707		763
708	Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. <i>IEEE Transactions on Pattern Analysis and Machine Intelligence</i> , 42(4):824–836.	764
709		765
710		766
711		767
712		768
713	Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On Faithfulness and Factuality in Abstractive Summarization. In <i>Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics</i> , pages 1906–1919.	769
714		770
715		771
716		772
717		773
718	Sewon Min, Kalpesh Krishna, Xinxu Lyu, et al. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In <i>Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing</i> , pages 12076–12100.	774
719		775
720		776
721		777
722		778
723		779
724	Ankur Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqi, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. 2020. ToTTo: A Controlled Table-To-Text Generation Dataset. In <i>Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing</i> , pages 1173–1186.	780
725		781
726		782
727		783
728		784
729		
730	Fabio Petroni, Patrick Lewis, Aleksandra Piktus, et al. 2021. How Context Affects Language Models’ Factual Predictions. In <i>Proceedings of the 1st Workshop on Fact Extraction and VERification</i> .	785
731		786
732		787
733		788
734	Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. 2025. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. In <i>Proceedings of the ACM Web Conference 2025</i> , pages 2366–2377.	789
735		790
736		791
737		792
738		793
739		794
740	Zackary Rackauckas. 2024. RAG-Fusion: A New Take on Retrieval Augmented Generation. <i>International Journal of Natural Language Computing</i> , 13(1):37–47.	795
741		796
742		797
743		798
744	Zhihong Shao, Yeyun Gong, Yelong Shen, et al. 2023. Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy. In <i>Findings of the Association for Computational Linguistics: EMNLP 2023</i> , pages 9248–9274.	799
745		800
746		801
747		802
748		803
		804
		805
	Liang Wang, Nan Yang, and Furu Wei. 2023. Query2Doc: Query Expansion with Large Language Models. In <i>Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing</i> , pages 9414–9423.	
	Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. 2024. Knowledge Graph Prompting for Multi-Document Question Answering. In <i>Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence</i> , pages 19206–19214.	
	Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. <i>Advances in Neural Information Processing Systems</i> .	
	Sam Wiseman, Stuart Shieber, and Alexander Rush. 2017. Challenges in Data-to-Document Generation. In <i>Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing</i> , pages 2253–2263.	
	Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhajan Kambadur, David Rosenberg, and Gideon Mann. 2023. BloombergGPT: A Large Language Model for Finance. <i>arXiv preprint arXiv:2303.17564</i> .	
	Qilong Wu, Xiaoneng Xiang, Hejia Huang, Xuan Wang, Yeo Wei Jie, Ranjan Satapathy, Ricardo Shirota Filho, and Bharadwaj Veeravalli. 2025. SusGen-GPT: A Data-Centric LLM for Financial NLP and Sustainability Report Generation. In <i>Findings of the Association for Computational Linguistics: NAACL 2025</i> , pages 1184–1203.	
	Miao Xie, Xiao Zhang, Yi Li, and Chunli Lv. 2026. Structure Guided Retrieval-Augmented Generation for Factual Queries. <i>ACL ARR 2026 January Submission</i> .	
	Zhilin Yang, Peng Qi, Saizheng Zhang, et al. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In <i>Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing</i> , pages 2369–2380.	
	Peiru Yang, Xintian Li, Zhiyang Hu, Jiapeng Wang, Jinhua Yin, Huili Wang, Lizhi He, Shuai Yang, Shanguang Wang, Yongfeng Huang, and Tao Qi. 2025. HeteRAG: A Heterogeneous Retrieval-Augmented Generation Framework with Decoupled Knowledge Representations. <i>arXiv preprint arXiv:2504.10529</i> .	
	Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. <i>Advances in Neural Information Processing Systems</i> .	
	Luyao Zhuang, Shengyuan Chen, Yilin Xiao, Huachi Zhou, Yujing Zhang, Hao Chen, Qinggang Zhang, and Xiao Huang. 2026. LinearRAG: Linear Graph Retrieval Augmented Generation on Large-scale Corpora. In <i>Proceedings of the International Conference on Learning Representations (ICLR 2026)</i> .	

A Symbol Definitions

Symbol	Description
$\mathcal{H}$	sample document collection
h_i	a single historical document
q	user query instruction
$\mathcal{D}$	data source collection
d_i	a single data source
s_i^j	the j -th state extracted from document h_i
$\text{Seq}(\cdot)$	state sequence extracted from a document or a document collection
θ	frequency threshold for state and transition filtering
$\text{freq}(\cdot)$	frequency function for a state or a state transition
L (or L_q)	DFA (or SubDFA), formalized as $(S, \Sigma, \delta, s_0, F)$ or $(S_q, \Sigma_q, \delta_q, s_{0q}, F_q)$
η	Document Logic Extraction
$e(s^j)$	semantic embedding vector of state s^j
$\mathcal{I}$	state index built over state embeddings
μ	Query Matching
$I(q)$	structured intent representation of query q
$\text{sim}(\cdot, \cdot)$	similarity function between query intent and state embeddings
τ	similarity threshold for state filtering in query matching
s_{0q}	root state identified for query q
Φ	State-Transition Generation
$C(s^j)$	retrieved state-level evidence for state s^j
$\text{Retrieve}(\mathcal{D}, q, \text{data}(s^j))$	state-level retrieval function
$\hat{y}^j$	generated text segment for state s^j
m^j	brief summary of $\hat{y}^j$ for next-state generation
$\hat{Y}$	final generated financial research report
T_{ret}	average retrieval cost for one state during online execution
T_{online}	total online inference complexity of LogicRAG
$\mathcal{F}$	embedding dimension of the state representation space
$ S $	number of states in the global DFA
$ S_q $	number of states in the task-specific sub-automaton
N_s	number of original states extracted from historical document

Table 5: Symbols and Descriptions.

B Heterogeneous Data Source Collection

This appendix provides a detailed description of the heterogeneous data source collection $\mathcal{D}$ introduced in the definition in Section 3, covering the design of the three-layer structured model of data sources, the data acquisition process, and real data examples.

B.1 Three-Dimensional Query Key

The heterogeneous data source collection $\mathcal{D}$ is organized with a **three-dimensional query key** mechanism to support the state-level retrieval function in Section 4.4:

$$\text{Retrieve}(\mathcal{D}, q, \text{data}(s^j)) \mapsto C(s^j).$$

Specifically, for any state s^j , the system does not retrieve directly by the state label. Instead, it combines $\text{data}(s^j)$ in the state quadruple with the time information in the user query q to jointly determine a set of structured query keys

$$(\text{CODES}, \text{INDICATORS}, \text{DATE}),$$

and uses them to locate the corresponding data source interface, indicator fields, and time range in $\mathcal{D}$, thereby completing state-level data acquisition.

Top-Level Identifier Key (the id Layer) The top-level identifier key is used to determine the **data source code** to be queried, namely CODES. Its role is to identify a concrete queryable object in the heterogeneous data source collection $\mathcal{D}$, such as the main continuous futures contract on an exchange. For a given state s^j , the system first reads

the data requirements defined in $\text{data}(s^j)$, such as “copper price”, “aluminum price”, and “gold futures price”; it then determine the specific data source code to access. For example, when $\text{data}(s^j)$ requires the “COMEX gold price”, the system can identify @GC0Y.CMX as the target CODES; if $\text{data}(s^j)$ requires the “SHFE gold main continuous price”, the corresponding code is AU00.SHF. In summary, CODES is determined by the product requirements defined in the state. Examples of data source codes are shown in Table 6.

Data Source Code (CODES)	Exchange	Product
ALI0Y.CMX	COMEX	Aluminum (main continuous)
@GC0Y.CMX	COMEX	Gold (main continuous)
@HG0Y.CMX	COMEX	Copper (main continuous)
AU00.SHF	SHFE	Gold (main continuous)
CU00.SHF	SHFE	Copper (main continuous)
AL00.SHF	SHFE	Aluminum (main continuous)

Table 6: Examples of data source codes in the top-level identifier layer

Content-Definition Key (the schema Layer)

The content-definition key is used to determine the **indicator fields** to be queried, namely INDICATORS. Unlike CODES, which determines “which data source to query”, INDICATORS determines “which fields to retrieve from that data source”. For any state s^j , the system parses the required data types and business variables from $\text{data}(s^j)$, such as “closing price”, “price change”, “settlement price”, “inventory change”, and “trading volume”, and maps these requirements to indicator codes in the interface. For example, if $\text{data}(s^j)$ describes a “price review”, it is more likely to map to ths_settle_future, ths_chg_future, and ths_chg_ratio_future; if it describes “trading volume and open-interest analysis”, it is more likely to map to ths_vol_future and ths_open_interest_future. Therefore, INDICATORS is essentially determined by the field requirements defined in $\text{data}(s^j)$. Some indicator codes and their business meanings are shown in Table 7.

Indicator Code (INDICATORS)	Business Meaning
ths_pre_settle_future	previous settlement price
ths_settle_future	current settlement price
ths_settle_chg_future	settlement change
ths_chg_future	closing-price change
ths_chg_ratio_future	price change (%)
ths_settle_chg_ratio_future	settlement change ratio (%)
ths_vol_future	trading volume
ths_open_interest_future	open interest

Table 7: Examples of indicator codes and names in the content-definition layer (excerpt)

Temporal-Dimension Key The temporal-dimension key is used to determine the **query time range**, namely DATE. This key is provided

by the time information in the user query q , such as “2025-02-28”, “March 10 to March 14, 2025”, “this week”, or “last week”. For example, if q specifies a weekly report task while $\text{data}(s^j)$ requires “weekly price change” or “week-end closing price”, the system parses the query time into the corresponding key dates within the week. If q explicitly provides a single-day date, that date is directly used as the DATE input to the interface.

B.2 Data Acquisition Process

$\mathcal{D}$ is obtained through the iFinD quantitative interface (iFinDPy), and the data acquisition process is shown in Algorithm 2.

Algorithm 2 Heterogeneous Data Source Acquisition Process

Input: iFinD account u , password p , data source codes CODES, indicators INDICATORS, query date DATE

Output: Structured data table $\mathbf{R}$

```

1:  $c \leftarrow \text{LOGIN}(u, p)$ 
2: if  $c \notin \{0, -201\}$  then
3:   return  $\emptyset$ 
4: end if
5:  $r \leftarrow \text{FETCH}(\text{CODES}, \text{INDICATORS}, \text{DATE})$ 
6: if  $r.\text{errorcode} \neq 0$  then
7:    $\text{LOGOUT}()$ 
8:   return  $\emptyset$ 
9: end if
10:  $\mathbf{R} \leftarrow \text{PARSE}(r.\text{data})$ 
11:  $\text{EXPORT}(\mathbf{R})$ 
12:  $\text{LOGOUT}()$ 
13: return  $\mathbf{R}$ 

```

Environment and Permission Dependencies

Running the above process requires the following conditions:

- **Python environment:** Python ≥ 3.8 , the dependency package iFinDPy (the official Tonghuashun SDK) and pandas $\geq 1.3.0$.
- **Interface permission:** a valid iFinD quantitative interface account is required, and futures market-data permissions for the target exchanges (COMEX and SHFE) must have been enabled.
- **Network requirement:** the server must be able to access iFinD data nodes, and deployment in a mainland China network environment is recommended.

B.3 Data Example

Table 8 shows examples of partial fields from records obtained by using DATE = 2025-02-28 as the query date and fetching six data source codes, with partial fields from the resulting data records. The complete field set contains 51 items (including night-session quotes, basis, open interest, and other dimensions).

No.	CODES	Prev. Settle	Curr. Settle	Settle Chg.	Close Chg.	Chg. Ratio (%)	Settle Chg. Ratio (%)	Volume	Open Interest
1	ALIBY_OH	2,596.25	2,561.50	-34.75	0.00	0.0000	-1.3385	10	153
2	BCGVY_OH	2,883.20	2,836.80	-46.40	-25.20	-1.2243	-1.6093	990	10,210
3	BHGVY_OH	4.58	4,5145	-0.0655	-0.0260	-1.0917	-1.4301	2,731	7,158
4	AU000_SHF	677.42	673.86	-3.56	-3.92	-0.7912	-0.5255	275	120
5	CU000_SHF	76,880.00	76,850.00	-30.00	-210.00	-0.2211	-0.0390	28,440	63,870
6	AL000_SHF	20,545.00	20,620.00	+75.00	+40.00	+0.4624	+0.3651	15,081	34,855

Table 8: Data example from the heterogeneous data source collection $\mathcal{D}$ (query date: 2025-02-28; partial fields shown).

C Detailed Algorithms of LogicRAG

C.1 Document Logic Extraction

Algorithm 3 Document Logic Extraction

Require: Sample document collection $\mathcal{H}$, frequency threshold θ

Ensure: DFA L , state index $\mathcal{I}$

```

1: for each document  $h_i$  in  $\mathcal{H}$  do
2:   Extract  $\text{Seq}(h_i)$   $\triangleright$  obtain the hierarchical state sequence
3: end for
4: Normalize all extracted states across  $\mathcal{H}$  to obtain  $\text{Seq}(\mathcal{H})$ 
 $\triangleright$  merge similar states into a shared state space
5: Compute  $\text{freq}(s)$  and  $\text{freq}(s^a \rightarrow s^b)$  from  $\mathcal{H}$ , and construct  $S$  and  $\delta$  according to  $\theta$   $\triangleright$  retain high-frequency states and transitions
6: Compute  $e(s^j) = \text{Embed}(\text{label}(s^j) \oplus \text{desc}(s^j))$  for each  $s^j \in S$ , and build  $\mathcal{I}$   $\triangleright$  build the index for online matching
7: Construct  $L = (S, \Sigma, \delta, s_0, F)$ 
8: return  $L, \mathcal{I}$ 

```

C.2 Query Processing

Algorithm 4 Query Processing

Require: User query q , DFA L , state index $\mathcal{I}$, similarity threshold τ

Ensure: Task-specific sub-DFA L_q

```

1: Parse  $q$  into  $I(q)$   $\triangleright$  obtain the structured intent representation
2: for each state  $s^j$  in  $S$  do
3:   Compute  $\text{sim}(I(q), e(s^j))$  and retain  $s^j$  if  $\text{sim}(I(q), e(s^j)) > \tau$   $\triangleright$  filter states by semantic relevance
4: end for
5: Identify the root state  $s_{0q}$  using the hierarchical parent-child relations  $\triangleright$  locate the root covering the retained states
6: Expand from  $s_{0q}$  along  $\delta$  until all retained states are covered, and obtain  $S_q$   $\triangleright$  ensure relevance and topological connectivity
7: Construct  $\delta_q, \Sigma_q$ , and  $F_q$  from  $S_q$ 
8: return  $L_q = (S_q, \Sigma_q, \delta_q, s_{0q}, F_q)$ 

```


C.3 State-Transition Generation

Algorithm 5 State-Transition Generation

Input: User query q , task-specific sub-DFA L_q , data source collection $\mathcal{D}$

Output: Generated document $\hat{Y}$

- 1: **for** each state s^j in S_q following the transition order in L_q **do**
- 2: Compute $C(s^j) = \text{Retrieve}(\mathcal{D}, q, \text{data}(s^j))$ $\triangleright$ retrieve state-level evidence
- 3: **end for**
- 4: Initialize $m^0 = \emptyset$
- 5: **for** each state s^j in S_q following the transition order in L_q **do**
- 6: Compute $\hat{y}^j = \text{LLM}(q, s^j, C(s^j), m^{j-1})$ $\triangleright$ generate text for state s^j
- 7: Compute $m^j = \text{LLM}(\hat{y}^j)$ $\triangleright$ summarize $\hat{y}^j$ for next-state generation
- 8: **end for**
- 9: Compute $\hat{Y}$ by concatenating all $\hat{y}^j$ according to the transition order in L_q
- 10: **return** $\hat{Y}$

D Prompt Templates

D.1 Document State Extraction

Section	Content
Goal	Given two domain-specific reports, identify all semantically meaningful original states and extract a shared hierarchical writing structure. Leaf states correspond to individual paragraphs, while upper-level states represent inductive summaries of their children.
Step 1: State Extraction	For each original state s_i^j , extract: • $\text{id}(s_i^j)$: unique identifier (e.g., "1", "1.1", "1.1.1") • $\text{label}(s_i^j)$: semantic label of the state • $\text{desc}(s_i^j)$: brief summary of the state content • state_type: one of {root, child, leaf} • level: depth in the hierarchical structure • parent, children: structural relations • content_guideline: writing style guidance for this state • length: character count (leaf: actual; non-leaf: sum of children) Format: (id) (label) (desc) (state_type) (level)
Step 2: Required Material Extraction	For each state, identify the required material as required material, i.e., the structured data requirement needed for generating this state. The extracted required material should be saved into $\text{data}(s_i^j)$ in the state quadruple. The material description should use abstract variable names (e.g., "price_change", "inventory_data", "macro_indicator") rather than specific numerical values.
Step 3: Output Format	Output a single valid JSON object with top-level fields: <code>template_id</code> , <code>language</code> , <code>template_description</code> , <code>structure_pattern</code> , <code>state_template</code> , <code>material_requirements_summary</code> , and <code>usage_instruction</code> . The JSON must be directly parseable by <code>json.loads()</code> without markdown wrapping or additional explanatory text.
Input Variables	<code>text1</code> : full text of Report 1 <code>text2</code> : full text of Report 2

Table 9: Prompt Template for Document State Extraction.

D.2 Query Intent Vector Parsing

Section	Content
Goal	Parse the query intent vector $I(q)$ from the user query q , capturing key dimensions such as report type, target security, and time range, to enable semantic node filtering and sub-automaton L_q construction from L^A .
Input	• {QUERY}: User's raw natural language query (e.g., "Please generate a stock research report on stock YY by broker XX for Q4 2024") • {NODE_LABEL_SET}: The full set of node labels in the DFA, provided as reference for intent matching.
Requirements	• Extract report type (e.g., "stock research report") • Extract target security (e.g., stock ticker or company name) • Extract time range (e.g., "Q4 2024") • Output structured intent fields; no subjective inference beyond the query
Output Example	{ "report_type": "stock research report", "target": "Stock YY", "time_range": "Q4 2024", "broker": "Broker XX" }
Output Format	A structured JSON object containing report type, target security, time range, and other intent fields, with no additional explanatory text.

Table 10: Prompt Template for Query Intent Vector Parsing.

D.3 State-Driven Report Generation

Section	Content
Goal	Generate the report segment $\hat{y}^j$ for the current state s^j based on the user query q , the state-level retrieved evidence $C(s^j)$, and the summary m^{j-1} passed from the previous state.
Input	• {QUERY}: User query, specifying the report theme, target object, and time scope • {STATE}: Current state information, including its semantic role, state description, writing style, and length requirement • {DATA_CONTEXT}: Multi-source retrieval results $C(s^j)$ bound to the current state • {PREV_SUMMARY}: Summary m^{j-1} generated from the previous state's output, used to maintain local coherence
Requirements	• Accurately incorporate all key numerical values and facts in $C(s^j)$ without omission • Maintain semantic continuity with the previous state through m^{j-1} while avoiding redundant repetition • Focus only on the current state's writing task and do not generate content belonging to other states • Output plain text only; Markdown formatting, titles, numbering, and metadata are prohibited
Output Example	"This week, COMEX gold closed at XX USD/oz, up XX% from last week, while domestic SHFE gold reached XX CNY/g, reflecting continued safe-haven demand under rising macro uncertainty..."
Output Format	A plain-text paragraph corresponding only to the current state s^j , with no titles, numbering, or metadata.

Table 11: Prompt Template for State-Driven Report Generation.

E Proofs of Time Complexity

E.1 Proof of Theorem 4.1

Proof. The offline stage corresponds to the document logic induction module η , and mainly includes four steps: state extraction, cross-document state normalization, frequency-driven automaton construction, and state semantic embedding with index construction.

First, in the state extraction stage, η needs to extract the hierarchical state sequence $\text{Seq}(h_i)$ from each document in the sample document collection $\mathcal{H}$, and record structural information such as state hierarchy depth, parent-child relations, and content length. Let T_h denote the average cost of extracting states and constructing the local logic structure for a single article. Then the complexity of executing this process for all documents in $\mathcal{H}$ is

$$O(|\mathcal{H}| \cdot T_h).$$

Second, in the cross-document state normalization stage, the system maps the state labels $\text{label}(s_i^j)$ extracted from each document into $\mathcal{F}$ -dimensional semantic vectors, and merges states with similar semantics and consistent functions through approximate nearest-neighbor retrieval. For N_s original states, if an approximate-nearest-neighbor index structure is used for cross-document matching, the matching cost for a single state can be approximated as $O(\mathcal{F} \cdot \log N_s)$. Therefore, the overall complexity of this stage is

$$O(N_s \cdot \mathcal{F} \cdot \log N_s).$$

Third, in the frequency-driven automaton construction stage, η needs to count the state occurrence frequency $\text{freq}(s)$ and the transition frequency of adjacent state pairs $\text{freq}(s^a \rightarrow s^b)$, and

filter the retained state set S and transition set δ according to the frequency threshold θ . Because both state frequencies and transition frequencies can be computed by traversing the state sequences and adjacent state pairs in historical documents, this process is linear in the number of original states. Therefore, its complexity is

$$O(N_s).$$

Finally, in the state semantic embedding and index construction stage, η jointly encodes the $\text{label}(s^j)$ and $\text{desc}(s^j)$ of each retained state $s^j \in S$ to obtain the state embedding $e(s^j)$, and builds the state index $\mathcal{I}$ based on these embeddings. If an approximate-nearest-neighbor index structure such as HNSW is used, the index construction complexity can be approximated as

$$O(|S| \cdot \mathcal{F} \cdot \log |S|).$$

Adding the above four components gives Equation (10). Because the offline stage only needs to be executed once when the knowledge base is constructed, its computation can be reused by multiple subsequent online queries, and therefore this cost does not directly increase the online inference cost of a single document generation run. $\square$

E.2 Proof of Theorem 4.2

Proof. The online stage mainly consists of query matching, state-by-state data retrieval, and state-driven generation.

First, in the query matching stage, μ encodes the user query q into the intent vector $I(q)$, and computes the similarity $\text{sim}(I(q), e(s^j))$ between the query intent and each state $s^j \in S$ in the deterministic finite automaton. Because each similarity calculation is performed in the $\mathcal{F}$ -dimensional embedding space, and this operation is executed once for all $|S|$ states, the complexity of similarity calculation is

$$O(|S| \cdot \mathcal{F}).$$

The system then performs one threshold-filtering decision over all states according to τ , whose complexity is

$$O(|S|).$$

Therefore, the total complexity of the query matching stage is

$$O(|S| \cdot (\mathcal{F} + 1)).$$

After obtaining the states that satisfy the threshold constraint, the system further combines the parent-child relations already extracted in η , searches in the deterministic finite automaton for a root state s_{0q} that has hierarchical parent-child relations with all these states, and expands reachable states from this root along the transition function δ until all retained states are covered, thereby obtaining the connected state set of the sub-automaton. This process only involves search over a local connected structure, and its complexity is at most linear in the final sub-automaton size $|S_q|$; therefore, it can be absorbed into the following linear execution term over $|S_q|$.

Second, in the state-by-state data retrieval stage, for each state $s^j \in S_q$, Φ constructs retrieval conditions based on $\text{data}(s^j)$ and the time information in query q , locates the target data source in the heterogeneous data source collection $\mathcal{D}$, and executes the query. If data-source location is regarded as an indexed lookup, its complexity can be approximated as $O(\log |\mathcal{D}|)$; if the cost of one actual API call is denoted as T_{api} , then the average retrieval complexity of a single state is

$$T_{\text{ret}} = O(\log |\mathcal{D}| + T_{\text{api}}).$$

Because this process is executed independently for each state in S_q , the overall complexity of state-by-state data retrieval is

$$O(|S_q| \cdot T_{\text{ret}}).$$

Finally, in the state-driven generation stage, for each state $s^j \in S_q$, the system calls the LLM based on the current state, the corresponding data instance $C(s^j)$, and the previous-state summary m^{j-1} to generate the text segment $\hat{y}^j$, and further generates the summary m^j to pass to the next state. Because this process performs a constant number of LLM calls for each state, let T_{LLM} denote the average LLM generation cost for a single state. Then the overall complexity of this stage is

$$O(|S_q| \cdot T_{\text{LLM}}).$$

Adding the complexities of query matching, state-by-state data retrieval, and state-driven generation gives Equation (11). $\square$

F FinLDP-Bench Dataset Construction

For each vertical category, we manually extract the main body of each original report and remove

privacy-related metadata such as author information. The extracted reports are stored in a CSV file, where each row corresponds to one report body. For each case in the vertical category, we randomly select two historical documents from the CSV file as the sample document collection $\mathcal{H}$, because the robustness analysis in Appendix L shows that LogicRAG achieves the best overall performance when the number of historical documents is set to 2. To increase task difficulty and evaluate robustness, partial paragraphs are removed with probability `prob_drop`, and the order of selected paragraphs is reversed with probability `prob_reverse`. Repeating this process produces 10,000 samples for each report category and 50,000 samples in total.

Algorithm 6 Construction of One Vertical Category in FinLDP-Bench

Input: CSV file containing report bodies of one vertical category, `prob_drop`, `prob_reverse`
Output: Sample document collections $\mathcal{H}$ for this vertical category

- 1: Read all report bodies from the CSV file
- 2: **for** each case to be constructed **do**
- 3: Randomly select two report bodies without replacement
- 4: **if** `Rand() < prob_drop` **then**
- 5: Remove partial paragraphs from the selected report bodies
- 6: **end if**
- 7: **if** `Rand() < prob_reverse` **then**
- 8: Reverse the order of selected paragraphs in the selected report bodies
- 9: **end if**
- 10: Save the processed two report bodies as the sample document collection $\mathcal{H}$
- 11: **end for**
- 12: **return** All constructed sample document collections $\mathcal{H}$ for this vertical category

G Additional Case Study on Precious Metals Report Generation

To more intuitively demonstrate the sample characteristics of the dataset, Table 12 presents a single report from a concrete sample in the Precious Metals vertical. This sample is composed by concatenating three randomly selected weekly tracking reports on the precious metals industry. One of them is a weekly report released by Soochow Securities Co., Ltd. on March 16, 2025 (report period: March 10–14, 2025). The report contains four fields: market review, sector view, sector tracking, and risk warning. The sector tracking field contains detailed data for three subcategories, copper, aluminum, and gold, involving more than 30 numerical citation points (including price changes,

absolute prices, inventory changes, and other values).

Field	Content
Market Review	The precious metals sector rose 3.56% this week (Mar. 10–14), ranking among the top gainers across all primary industries. Among sub-sectors, small metals led with +4.86%, followed by precious metals (+4.67%), energy metals (+3.93%), industrial metals (+3.64%), and new metallic materials (−1.43%).
Sector View	Industrial metals: the Fed is expected to restart rate cuts in mid-year (50–75 bp), with domestic downstream entering peak season driving broad-based gains amid declining social inventory. Precious metals: Michigan consumer sentiment declined further in March while long-term inflation expectations rose, reinforcing stagflation pricing; Comex gold broke through the key level of \$3,000/oz.
Sector Tracking	Copper: LME copper closed at \$9,793/t (+1.99% WoW); SHFE copper at 80,500 CNY/t (+2.78% WoW). Import TC fell to −16 USD/t; Tongling smelter announced maintenance and additional output cuts. Aluminium: LME aluminium at 2,689 USD/t (−0.02% WoW); SHFE aluminium at 20,990 CNY/t (+0.74% WoW). Theoretical operating capacity reached 43,725 Mt (+0.125 Mt WoW); social inventory fell to 88.70 Mt (−1.38 Mt WoW). Gold: COMEX gold closed at \$2,993.60/oz (+2.60% WoW); SHFE gold at 694.96 CNY/g (+2.28% WoW). US Feb. CPI +2.8% YoY (est. 2.9%); PPI +3.2% YoY (est. 3.3%).
Risk Warning	Sustained USD strength; downstream demand below expectations.

Table 12: A sample report excerpt from the Precious Metal subset (Soochow Securities, Mar. 16, 2025; report period: Mar. 10–14, 2025). The structured output fields are derived from a single report excerpt and must cite all numerical values appearing in the source text.

H Generalization Experiment on RotoWire

To further verify the generalization ability of LogicRAG beyond financial report generation, and to examine the applicability of its state-transition generation mechanism to general structured data generation tasks, this paper conducts generalization experiments on the RotoWire dataset. RotoWire is a classic benchmark for structured data-to-text generation, where the input consists of team and player statistics from basketball games and the output is the corresponding game news summary. Unlike FinLDP-Bench, the core task of RotoWire is not financial-domain writing-logic induction, but selecting key facts from structured game data and organizing them into coherent text. To adapt to this task, we interpret game reports as state sequences composed of functional states such as game result, key player performance, team-level performance, game process, and post-game summary, and compare four methods: Direct LLM, Naive RAG, LogicRAG w/o DFA, and full LogicRAG. This tests whether state-transition constraints can improve fact coverage, content organization, and discourse consistency in structured-data-driven generation scenarios.

Method	BLEU	ROUGE-L	Fluency	Consistency	ACC
Direct LLM	10.14	23.75	75.8	69.3	55.28
Naive RAG	17.38	24.06	77.0	69.7	77.39
LogicRAG w/o DFA	17.32	26.11	72.7	69.7	74.67
LogicRAG	18.82	27.23	77.1	70.5	92.30

Table 13: Generalization experiment results on the RotoWire dataset

As shown in Table 13, Direct LLM performs reasonably in Fluency, but its ACC is only 55.28, indicating that relying solely on an LLM to directly read structured data makes it difficult to stably cover key facts in game reports. Naive RAG raises ACC to 77.39 by introducing similar game reports as contextual examples, showing that example-driven generation can improve factual expression and text quality. LogicRAG w/o DFA reaches 26.11 in ROUGE-L, higher than Naive RAG’s 24.06, indicating that a structured writing template helps improve content organization; however, without explicit state-transition constraints, its gains in ACC and Consistency are limited. In contrast, full LogicRAG achieves the best performance on all five metrics, with ACC reaching 92.30, clearly higher than Naive RAG and LogicRAG w/o DFA, while BLEU, ROUGE-L, Fluency, and Consistency also remain the highest. This shows that LogicRAG’s state-transition generation mechanism can not only improve logical consistency in financial report generation, but can also transfer to structured data-to-text generation tasks, effectively enhancing fact coverage, content organization, and discourse coherence, thereby demonstrating good cross-task generalization.

I Evaluation Protocol and Metrics

Qualitative Evaluation

Because LLMs have inherent judging preferences over generated content (Zheng et al., 2023), this study adopts the multi-model independent judging paradigm in the G-EVAL evaluation framework (Liu et al., 2023). Three independent LLMs are used for evaluation, and their arithmetic mean is taken as the final score to ensure objectivity and fairness. The evaluation metrics include Fluency and Consistency, which measure the naturalness of language and structural consistency, respectively. Both metrics use the reference document as the benchmark, with a full score of 100, and adopt a two-stage scoring mechanism consisting of “level interval + fine-grained dimensions”. Specifically, we first determine the level interval according to the overall difference between the generated document and the reference document and assign a base score, then score each fine-grained dimension within that interval and add the offset.

Fluency: This metric measures the similarity between the generated document and the reference document at the level of language expression, fo-

cusing on consistency in word choice, sentence structure, tone and voice, and expression habits. The level intervals and corresponding base scores are shown in Table 14:

Level	Base Score	Description
Level 5	80	Language expression is fully consistent with the reference document.
Level 4	60	Highly consistent, with only minimal differences.
Level 3	40	Basically consistent, but with noticeable differences in expression.
Level 2	20	Obvious differences in language style from the reference document.
Level 1	0	Language expression deviates greatly from the reference document.

Table 14: Fluency level definitions and base scores.

After determining the level interval, the following four fine-grained dimensions are scored one by one (0–5 points each). The sum of these scores is used as the offset within the interval and added to the base score to obtain the final Fluency score:

- **Appropriateness of Word Choice:** evaluates whether professional terminology and word choices are consistent with the reference document, considering the standardization and precision of domain vocabulary.
- **Accuracy of Person Usage:** evaluates whether the choice of narrative perspective, such as first or third person, matches the reference document, considering its influence on textual coherence.
- **Match of Expression Habits:** evaluates whether idiomatic collocations, fixed expressions, and domain writing habits are consistent with the reference document, reflecting stylistic alignment.
- **Overall Reading Fluency:** comprehensively evaluates whether the generated text is natural, fluent, and coherent.

Consistency: This metric measures the similarity between the generated document and the reference document at the framework-structure level, focusing on section order, content completeness, emphasis allocation, and hierarchical organization, and reflecting whether the generated text follows domain-specific writing logic. The level intervals and corresponding base scores are shown in Table 15:

Level	Base Score	Description
Level 5	80	Framework structure is almost completely consistent with the reference document.
Level 4	60	Overall framework is consistent, with only minor structural deviations.
Level 3	40	Structure is broadly similar, but differences are noticeable.
Level 2	20	Structure is inconsistent, affecting the overall reading logic of the document.
Level 1	0	Framework structure severely deviates from the reference document.

Table 15: Consistency level definitions and base scores.

After determining the level interval, the following four fine-grained dimensions are scored one by one (0–5 points each). The sum of these scores is used as the offset within the interval and added to the base score to obtain the final Consistency score:

- **Content Completeness:** evaluates whether the key content sections in the reference document are fully covered in the generated document, measuring the degree of information omission.
- **Reasonableness of Detail Hierarchy:** evaluates whether the arrangement of detailed descriptions and summaries in each part is consistent with the reference document, considering alignment in information granularity.
- **Normativity of Hierarchical Structure:** evaluates whether the hierarchical affiliation between the body text and subsections is correct, reflecting the regularity of the overall document organization.
- **Naturalness of Paragraph Transitions:** evaluates whether transitional expressions and transition logic between sections are consistent with the reference document, considering discourse coherence.

Quantitative Evaluation. This study introduces four widely used objective metrics (Guo et al., 2024), ACC (data accuracy), BLEU-P, ROUGE-L, and F1, to comprehensively evaluate generated documents from the two dimensions of numerical accuracy and text matching quality. The evaluation is defined as follows:

$$ACC = \frac{\sum_{i=1}^N \mathbf{1}[\hat{d}_i \in \mathcal{D}_{gen}]}{N} \quad (12)$$

$$BLEU-P = \frac{\sum_{s \in \mathcal{S}_{gen}} \text{Count}_{clip}(s)}{\sum_{s \in \mathcal{S}_{gen}} \text{Count}(s)} \quad (13)$$

$$ROUGE-L = \frac{LCS(\mathcal{S}_{gen}, \mathcal{S}_{ref})}{|\mathcal{S}_{ref}|} \quad (14)$$

$$F1 = \frac{2 \cdot BLEU-P \cdot ROUGE-L}{BLEU-P + ROUGE-L} \quad (15)$$

Here, in Equation (12), N is the total number of data points in the reference text, $\hat{d}_i$ is the i -th reference data point, and $\mathcal{D}_{gen}$ is the set of data points in the generated text. Equation (13) follows the n-gram clipped-count mechanism of BLEU. $\text{Count}_{clip}(s)$ is the effective number of matches in the reference text for each n-gram fragment s in the generated text; when the occurrence count of a fragment in the generated text exceeds its occurrence count in the reference text, the excess is not counted, ensuring strict exact matching. Equation (14) follows the longest common subsequence mechanism of ROUGE-L, measuring the coverage of reference content and sequential structure by the ratio between the longest common subsequence length of the generated and reference texts and the total length of the reference text. Here, $LCS(\mathcal{S}_{gen}, \mathcal{S}_{ref})$ is the length of the longest common word sequence that preserves relative order between the two, and $|\mathcal{S}_{ref}|$ is the total length of the reference text. In Equation (15), F1 is the harmonic mean of BLEU-P and ROUGE-L, comprehensively reflecting the overall matching quality of the generated text.

J Efficiency Analysis

As shown in Table 16, LogicRAG’s average offline construction time is **66.98s**, slower than NaiveRAG, CRAG, and HeterRAG, but faster than GraphRAG. This overhead mainly comes from document logic extraction, state normalization, and automaton construction. Since the offline stage only needs to be constructed once and does not require frequent rebuilding, this cost is acceptable. As shown in Table 17, LogicRAG’s average online inference time is **43.57s**, slightly slower than NaiveRAG, GraphRAG, and HeterRAG, but much faster than CRAG. Overall, the efficiency gap is moderate, and this trade-off is worthwhile because LogicRAG directly produces a complete report with stronger logic consistency and numerical accuracy.

Table 16: Average offline time (in seconds) of LogicRAG and representative baselines across subsets of FinLDP-Bench.

Methods	Macro	Cotton	Precious Metals	ETF	Agriculture	Average
NaiveRAG	9.76	5.06	4.73	6.36	7.58	6.70
HeterRAG	33.17	17.19	16.09	21.64	25.78	22.77
CRAG	18.54	9.61	8.99	12.09	14.41	12.73
GraphRAG	112.20	58.64	54.90	73.82	88.30	77.57
LogicRAG	97.57	50.56	47.32	63.64	75.82	66.98

Table 17: Average online inference time (in seconds) of LogicRAG and representative baselines across subsets of FinLDP-Bench.

Methods	Macro	Cotton	Precious Metals	ETF	Agriculture	Average
NaiveRAG	44.06	28.18	23.89	27.61	61.42	37.03
HeteRAG	49.25	31.49	26.70	30.86	68.65	41.39
CRAG	147.74	94.48	80.11	92.57	205.94	124.17
GraphRAG	40.96	26.19	22.21	25.66	57.09	34.42
LogicRAG	51.84	33.15	28.11	32.48	72.26	43.57

K Additional Case Study on ICE U.S. Cotton Price Generation

To further analyze the actual output quality of different methods in the logic-driven document generation task, this section provides a supplementary case from the Cotton vertical. Specifically, we use the following user query:

Please write the relevant information for November 21, 2025 about ICE U.S. cotton futures prices, the front-month contract closing price, and the week-over-week change.

The reference text corresponding to this query is:

ICE U.S. cotton futures prices fluctuated and declined; the front-month March contract closed at 63.93 cents/lb, down 0.21 cents/lb from the previous week, a decline of 0.33%.

Table 18 shows the actual outputs of LogicRAG and the comparison baselines for the above query, and compares them one by one with the reference text. We analyze the output quality of different methods from the perspectives of contract month, closing price, weekly absolute change, percentage decline, and time-range consistency.

Model	Relevance	Extracted Content & Note
Qwen2.5-Max	Partial match	The front-month Dec. ICE cotton contract closed at 64.54 cents/lb, down approx. 1.97% WoW. <i>Note: Contract month (Dec.) and price (64.54 cents/lb) diverge from reference; direction consistent.</i>
DeepSeek-V3	Partial match	The ICE U.S. cotton front-month March contract closed at 63.93 cents/lb, down 0.21 cents/lb WoW, a decline of 0.33%. <i>Note: Numerical fields match; however, trend analysis attributes the move to export demand recovery, inconsistent with the actual driver (supply-side pressure).</i>
GPT-4o	Partial match	The ICE U.S. cotton front-month contract closed at around 84.50 cents/lb as of Jan. 26, up approx. 0.65% WoW. <i>Note: Wrong time period (January) and opposite price direction; severely inconsistent with reference.</i>
Doubao 2.0	Not mentioned	No mention of ICE U.S. cotton. Output focuses on corn and soybean market outlook.
Naive RAG	Not mentioned	Only mentions USD index movements, weather in major producing countries, and weekly export data. No specific price figures retrieved.
ITER-RETGEN	Not mentioned	Only references futures spread structures (5–9 and 9–1 spreads) and sell-on-rally strategy. No ICE cotton price retrieved.
GraphRAG	Partial match	As of June 20 close, ICE U.S. cotton futures fluctuated lower; the front-month December contract closed at 66.76 cents/lb, a decline of 1.68%. <i>Note: Contract month (Dec.), date (June 20), price (66.76 cents/lb) and decline magnitude (1.68%) all deviate from reference; only price direction is consistent.</i>
HeteRAG	Partial match	As of March 21 close, ICE U.S. cotton futures fell sharply; the front-month May contract closed at 65.35 cents/lb, a decline of 2.94%. <i>Note: Contract month (May), date (Mar. 21), price (65.35 cents/lb) and decline magnitude (2.94%) all diverge from reference; price direction is correct but numerical deviation is significant.</i>
LogicRAG	Exact match	ICE U.S. cotton futures fluctuated and declined; the front-month March contract closed at 63.93 cents/lb, down 0.21 cents/lb WoW, a decrease of 0.33%. <i>Note: Contract month, closing price, weekly change, and percentage all fully consistent with reference.</i>

Table 18: Case study results on ICE U.S. cotton price generation across baseline methods and the proposed LogicRAG.

In the Relevance column, “Exact match” denotes full consistency with the reference in terms of contract month, closing price, weekly change, and percentage decline; “Partial match” denotes correct directional or categorical identification with divergence in one or more numerical fields; and “Not mentioned” denotes that no ICE U.S. cotton price information is generated.

Table 18 shows clear differences among methods in this case. Qwen2.5-Max can identify the downward direction of ICE U.S. cotton prices, but shifts across periods in both contract month and closing price. DeepSeek-V3 matches the reference text in numerical fields, but its explanation of the price-movement driver is inconsistent with the actual market logic. GPT-4o shows clear time-period confusion: it not only cites data from the wrong period, but also judges the price trend as rising. Doubao 2.0, Naive RAG, and ITER-RETGEN all fail to generate the target price information, indicating clear limitations in fine-grained target-content localization. In contrast, although GraphRAG and HeteRAG can identify the overall downward direction of ICE U.S. cotton prices, they deviate from the reference text to varying degrees in key fields such as date, contract month, closing price, and decline magnitude. This indicates that even baselines with strong retrieval ability are still prone to cross-period confusion and numerical shifts in financial document generation scenarios involving explicit time constraints and precise numerical fields.

LogicRAG’s output is fully consistent with the reference text in the four key fields of contract month, closing price, weekly absolute change, and percentage decline, and the report date accurately corresponds to November 21, 2025 as required by the query. This case shows that LogicRAG can more effectively locate the required content according to the writing logic of the target document, and can maintain consistency in key financial values and time ranges during generation, thereby demonstrating stronger reliability and accuracy in fine-grained financial document generation tasks.

L Additional Analysis on Scalability and Robustness

This section verifies the scalability of LogicRAG from the two dimensions of data scale and text complexity, and explores the influence of the number of reference samples on model performance through robustness analysis.

Effect of Data Scale. To evaluate the influence of the number of data fields on system performance, this study divides FinLDP-Bench samples into five intervals uniformly according to the number of data fields from small to large, with each interval accounting for 20% of all samples, and evaluates the generation performance of LogicRAG on each interval. The results are shown in Table 19 and Figure 5 (left). As the number of data fields increases, all metrics fluctuate only slightly and do not show systematic degradation with increasing field count, indicating that LogicRAG has good scalability to changes in data scale.

Table 19: Scalability analysis of LogicRAG across sample groups ordered by number of data fields from small to large (each group contains 20% of the total samples).

Group	Fluency	Consistency	ACC	BLEU-P	ROUGE-L	F1
Q1 (0–20%)	61.1	72.6	70.3	40.4	50.9	44.9
Q2 (20–40%)	58.0	63.8	76.9	35.4	46.5	40.0
Q3 (40–60%)	56.9	64.0	78.4	34.7	45.4	39.1
Q4 (60–80%)	55.5	61.1	76.4	33.9	43.6	37.9
Q5 (80–100%)	55.2	61.2	73.8	33.9	42.7	37.6

Effect of Text Complexity. To explore the influence of input text length on system performance, this study divides samples uniformly into five intervals according to total text length from short to long, with each interval accounting for 20% of all samples. The results are shown in Table 20 and Figure 5 (right). All metrics remain at similar levels as text length increases. Even in the longest-text Q5 interval, no clear performance drop appears, indicating that LogicRAG adapts stably to changes in input text complexity. Taken together, the two groups of experiments show that LogicRAG exhibits consistently stable generation performance across both data scale and text complexity, demonstrating good scalability.

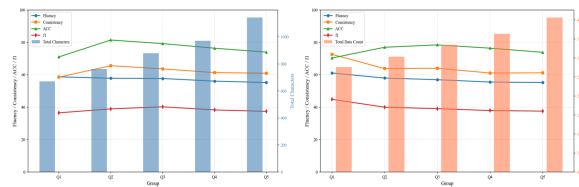


Figure 5: Scalability analysis of LogicRAG under varying data scale (left) and text complexity (right). Each group contains 20% of the total samples. Bar charts indicate the average total character count (left) and average data field count (right) per group.

Table 20: Scalability analysis of LogicRAG across sample groups ordered by total text length from short to long (each group contains 20% of the total samples).

Group	Fluency	Consistency	ACC	BLEU-P	ROUGE-L	F1
Q1 (0–20%)	58.7	58.5	71.0	32.0	42.9	36.5
Q2 (20–40%)	57.8	65.6	81.4	34.0	46.0	38.9
Q3 (40–60%)	57.7	63.6	79.2	35.9	46.3	40.2
Q4 (60–80%)	56.1	61.3	76.4	34.3	43.9	38.3
Q5 (80–100%)	55.2	60.9	74.0	33.8	42.5	37.4

Robustness Analysis. To explore the influence of the number of historical samples on model performance, we conduct experiments on FinLDP-Bench for $k \in \{1, 2, 3, 4, 5\}$ one by one. The results are shown in Table 21. Overall, all metrics fluctuate only slightly as the number of reference samples changes, indicating that LogicRAG is robust to the choice of reference-sample count. When $k = 2$, the model achieves the best or near-best overall performance on core metrics: ACC reaches 72.4, while BLEU-P and ROUGE-L are 32.3 and 41.8, respectively, all outperforming other parameter settings. When $k \geq 3$, these metrics decline to some extent, possibly because too many reference samples introduce mutually conflicting state types and transition conditions, interfering with template induction of the target writing logic. Considering both performance and computational cost, this study finally selects $k = 2$ as the default parameter setting.

Table 21: Robustness analysis of the number of in-context reference samples on FinLDP-Bench. We report the metrics of Fluency (%), Consistency (%), ACC (%), BLEU-P (%), ROUGE-L (%), and F1 (%), respectively.

Samples	Fluency	Consistency	ACC	BLEU-P	ROUGE-L	F1
1	67.8	59.5	66.8	15.7	25.8	19.6
2	64.8	62.0	72.4	32.3	41.8	36.4
3	65.5	58.9	66.1	26.2	36.6	30.5
4	67.6	61.5	70.3	17.1	37.4	23.5
5	65.0	59.3	70.3	15.2	35.5	21.3